Course Project Report
Assembly-Level Execution on Integrated RISC-V Processor

Zoya Imran (10204), Zaara Talat Jabeen (10196)

Spring 2026

Contents

1 Introduction **3**

2 Task A: Execution of Countdown Assembly Program **3**
2.1 Objective . . . 3
2.2 Assembly Program Functionality . . . 4
2.3 Processor Execution Steps . . . 4
2.4 Observed Results . . . 5
2.5 Waveform Analysis (Task A) . . . 5
2.6 Key Concepts Demonstrated . . . 5

3 Task B: Instruction Set Extension **6**
3.1 Instruction 1: SLT . . . 6
3.1.1 Description . . . 6
3.1.2 Datapath Changes . . . 6
3.1.3 Verification . . . 6
3.2 Instruction 2: BGE . . . 7
3.2.1 Description . . . 7
3.2.2 Control Path Changes . . . 7
3.2.3 Verification . . . 7
3.3 Instruction 3: LUI . . . 7
3.3.1 Description . . . 7
3.3.2 Datapath Modifications . . . 7
3.3.3 Verification . . . 7
3.4 Assembly-Level Execution Explanation . . . 7
3.4.1 SLT Execution Flow . . . 8
3.4.2 BGE Execution Flow . . . 8
3.4.3 LUI Execution Flow . . . 9
3.4.4 Delay Mechanism . . . 9
3.5 Processor-Level Observations . . . 9
3.6 Task B Summary Table . . . 10
3.7 Waveform Analysis (Task B) . . . 10

4 Task C: Fibonacci Series Execution **10**
4.1 Objective 10
4.2 Program Functionality 11
4.3 Subroutine Usage 11
4.4 Observed FPGA Output 11
4.5 Assembly-Level Execution Explanation 12
4.5.1 Initialization 12
4.5.2 Main Loop Execution 12
4.5.3 Fibonacci Computation 12
4.5.4 Loop Control 12
4.5.5 Subroutine Execution 13
4.5.6 NOP Instructions 13
4.6 Processor Execution Behavior 13
4.7 Waveform Analysis (Task C) 13
4.8 Key Concepts Demonstrated 14
4.9 Task C Summary 14

5 Conclusion **14**
5.1 Major Achievements 14
5.2 Overall Learning Outcomes 15

1 Introduction

This project focused on implementing and executing assembly-level programs on a custom-designed single-cycle RISC-V processor developed throughout previous laboratory sessions. The objective was to integrate all independently developed hardware modules into a complete functioning processor system, execute real assembly programs, extend the architecture with new instructions, and validate processor functionality through simulation and FPGA deployment.

The processor was constructed using several essential RTL modules:

- Program Counter (PC)
- Instruction Memory
- Register File
- Arithmetic Logic Unit (ALU)
- ALU Control Unit
- Main Control Unit
- Immediate Generator
- Data Memory
- Address Decoder
- Clock Divider
- Seven Segment Display Module

In Lab 11, these modules were integrated into a top-level processor design capable of executing a subset of RISC-V instructions. This project built upon that foundation by implementing practical assembly programs, introducing new instruction support, and verifying complete datapath functionality.

The project was divided into three major tasks:

- Task A: Execution of FSM-based countdown assembly program
- Task B: Implementation of three additional instructions (SLT, BGE, LUI)
- Task C: Execution of Fibonacci sequence generation program

Through these tasks, the project demonstrated processor design, datapath understanding, control path modification, memory-mapped I/O interaction, stack handling, subroutine execution, and instruction-level architecture extension.

2 Task A: Execution of Countdown Assembly Program

2.1 Objective

Task A required loading and executing an FSM-style countdown assembly program on the integrated RISC-V processor. The program reads user input from FPGA switches and displays a countdown sequence on LEDs.

2.2 Assembly Program Functionality

The assembly program performs the following steps:

1. Initializes registers:
   - x20 = initial test value
   - x2 = stack pointer
   - x5 = switch input address (0x300 / 768)
   - x6 = LED output address (0x200 / 512)
2. Clears LEDs
3. Continuously polls switch input using:

```
lw x14, 0(x5)
beq x14, x0, main_loop
```

4. Once a non-zero switch value is detected:
   - Passes input value to countdown subroutine
   - Calls subroutine using `jal`
5. Countdown subroutine:
   - Saves return address and temporary registers on stack
   - Displays countdown on LEDs
   - Uses delay loops for visibility
   - Decrements value until zero
   - Restores stack
   - Returns using `jalr`

2.3 Processor Execution Steps

1. Assembly program converted to machine code
2. Machine code stored in `program.mem`
3. Instruction memory loaded using `$readmemh`
4. Testbench simulated switch inputs
5. Datapath executed instructions sequentially
6. Outputs observed via:
   - LEDs
   - Seven-segment display
   - Waveform simulation

2.4 Observed Results

Example:

- Switch input = 3
- LEDs output:

$$3 \rightarrow 2 \rightarrow 1 \rightarrow 0$$

2.5 Waveform Analysis (Task A)

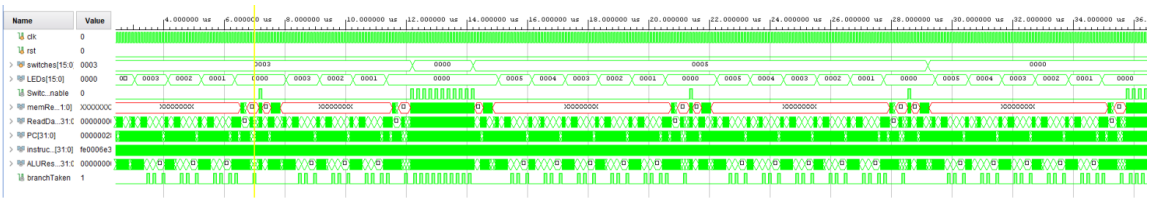

Figure 1: Task A Simulation Waveform

Explanation:

- PC increments sequentially during execution
- Branch instruction keeps looping until switch input is non-zero
- LED values decrease step-by-step showing countdown
- Memory write signals correspond to LED updates

2.6 Key Concepts Demonstrated

- Memory-mapped I/O
- Branching (BEQ/BNE)
- Stack operations
- Subroutine calling
- Delay loops
- Register preservation

3 Task B: Instruction Set Extension

Task B focused on extending the functionality of the custom RISC-V processor by adding support for additional instructions that were not originally implemented in the baseline datapath. The purpose of this task was to demonstrate how processor architectures can be expanded through modifications in the ALU, control unit, immediate generator, and branch logic.

The implementation of these instructions required understanding of instruction formats, control signal generation, ALU operation selection, and datapath execution flow. After implementation, dedicated assembly programs were executed to verify the correctness of each instruction using FPGA outputs and simulation waveforms.

Task B involved extending the processor's ISA by implementing three new instructions:

- SLT (Set Less Than)
- BGE (Branch if Greater or Equal)
- LUI (Load Upper Immediate)

3.1 Instruction 1: SLT

3.1.1 Description

SLT compares two registers:

$$rd = (rs1 < rs2)?1 : 0$$

3.1.2 Datapath Changes

- Added new ALU operation:

```
ALUResult = ($signed(A) < $signed(B)) ? 32'd1 : 32'd0;
```

- Updated ALUControl:

```
funct3 = 010 -> ALUControl = 0111
```

3.1.3 Verification

- 5 ¡ 10 produced LED output:

$$0001$$

- 10 ¡ 5 produced:

$$0000$$

3.2 Instruction 2: BGE

3.2.1 Description

Branches when:

$$rs1 \geq rs2$$

3.2.2 Control Path Changes

Modified branch logic:

```
(funct3 == 3'b101 & ~ALUResult[31])
```

3.2.3 Verification

- True case displayed:

$$00AA$$

- False case displayed:

$$00FF$$

3.3 Instruction 3: LUI

3.3.1 Description

Loads upper 20 bits:

$$rd = imm << 12$$

3.3.2 Datapath Modifications

- Added LUI signal in MainControl
- Modified writeback path:

```
regWriteData = (LUI) ? imm : writeBackData;
```

3.3.3 Verification

Instruction:

```
lui x14, 0xABCDE
```

Observed:

$$E000$$

3.4 Assembly-Level Execution Explanation

The implemented instructions were tested using a structured assembly program that executes sequentially and verifies each instruction through LED output.

3.4.1 SLT Execution Flow

```
addi x7, x0, 5
addi x8, x0, 10
slt x14, x7, x8
sw x14, 0(x6)
```

Execution:

- x7 is initialized to 5 and x8 to 10
- SLT compares the two values
- Since 5 ¡ 10, x14 is set to 1
- The result is written to LED memory address (512)

Second case:

```
slt x14, x8, x7
sw x14, 0(x6)
```

- 10 ¡ 5 is false
- x14 is set to 0
- LED displays 0000

3.4.2 BGE Execution Flow

```
addi x7, x0, 10
addi x8, x0, 5
addi x14, x0, 0xAA

bge x7, x8, bge_pass1
addi x14, x0, 0xFF
bge_pass1:
sw x14, 0(x6)
```

- x7 = 10 and x8 = 5
- Condition (10 5) is true
- Branch is taken
- x14 remains 0xAA and is displayed

Second case:

```
addi x14, x0, 0xFF
bge x8, x7, bge_pass2
sw x14, 0(x6)
```

- Condition (5 10) is false
- Branch not taken
- x14 = 0xFF is displayed

3.4.3 LUI Execution Flow

```
lui x14, 0xABCDE
sw x14, 0(x6)
```

- LUI loads immediate into upper 20 bits
- x14 = 0xABCDE000
- Only lower 16 bits are visible on LEDs
- Display shows E000

3.4.4 Delay Mechanism

```
addi x15, x0, 5
delay:
addi x15, x15, -1
bne x15, x0, delay
```

- Creates visible delay between outputs
- Ensures correct observation on FPGA

3.5 Processor-Level Observations

- The ALU performed arithmetic comparison operations for SLT and BGE.
- The Main Control Unit generated appropriate control signals based on opcode decoding.
- The ALU Control module selected the correct ALU operation using funct3 and funct7 fields.
- Branch instructions modified the Program Counter when branch conditions evaluated to true.
- LUI bypassed normal ALU computation and directly loaded the shifted immediate value into the destination register.
- Results were successfully written back into the register file and displayed through memory-mapped FPGA outputs.

The successful execution of these instructions demonstrated that the processor datapath was modular and could be extended with additional ISA functionality.

3.6 Task B Summary Table

Instruction	Expected Output	Observed Output
SLT (5¡10)	0001	0001
SLT (10¡5)	0000	0000
BGE True	00AA	00AA
BGE False	00FF	00FF
LUI	E000	E000

3.7 Waveform Analysis (Task B)

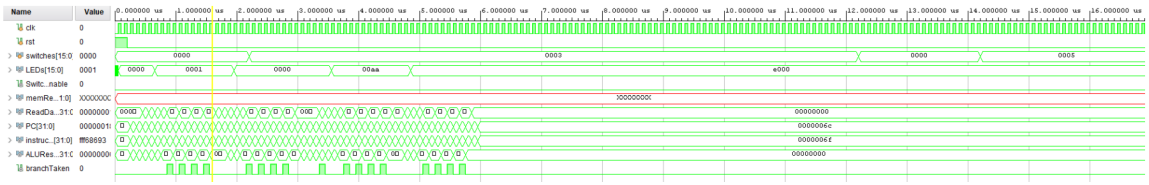

Figure 2: Task B Simulation Waveform

Explanation:

- ALU performs comparison operations for SLT
- Branch signal activates correctly for BGE
- Register values change based on instruction execution
- LED outputs match expected results for each instruction

4 Task C: Fibonacci Series Execution

Task C involved implementing and executing a Fibonacci sequence generator using assembly language on the custom RISC-V processor. This task demonstrated the processor's capability to execute iterative arithmetic programs involving loops, branching, stack operations, subroutine calls, and memory-mapped I/O.

The Fibonacci sequence was selected because it requires repeated arithmetic computation and continuous register updates, making it useful for validating datapath execution, ALU functionality, control flow handling, and processor state transitions.

The generated Fibonacci values were displayed on FPGA outputs using the memory-mapped LED interface, while waveform simulations were used to verify correct instruction execution and register updates.

4.1 Objective

Task C implemented a Fibonacci sequence generator on the processor.

4.2 Program Functionality

The program:

1. Initializes:
   - prev = 0
   - curr = 1
   - count = 10
2. Executes loop:
   - Displays current Fibonacci term
   - Calls display subroutine
   - Computes next term:

$$next = prev + curr$$

   - Updates:

$$prev = curr$$

$$curr = next$$

3. Stops after 10 terms

4.3 Subroutine Usage

The display subroutine:

- Saves x1 and x10 on stack
- Displays term on LEDs
- Performs delay
- Restores registers
- Returns using `jalr`

4.4 Observed FPGA Output

$$1 \to 1 \to 2 \to 3 \to 5 \to 8 \to 13 \to 21 \to 34 \to 55$$

Hexadecimal LED output:

$$1,\ 1,\ 2,\ 3,\ 5,\ 8,\ D,\ 15,\ 22,\ 37$$

4.5 Assembly-Level Execution Explanation

The Fibonacci program executes iteratively using registers and a subroutine to display results.

4.5.1 Initialization

```
addi x7, x0, 0
addi x8, x0, 1
addi x15, x0, 10
```

- x7 stores previous value (0)
- x8 stores current value (1)
- x15 is loop counter

4.5.2 Main Loop Execution

```
add x10, x8, x0
jal x1, display_and_delay
```

- Current Fibonacci value copied into x10
- Subroutine is called using jal
- Return address stored in x1

4.5.3 Fibonacci Computation

```
add x9, x7, x8
add x7, x8, x0
add x8, x9, x0
```

- next = prev + curr
- prev updated to curr
- curr updated to next

4.5.4 Loop Control

```
addi x15, x15, -1
bne x15, x0, fib_loop
```

- Loop continues until counter becomes zero

4.5.5 Subroutine Execution

```
sw x10, 0(x6)
addi x16, x0, 5
```

- Value displayed on LEDs
- Delay loop executed

```
jalr x0, 0(x1)
```

- Returns to main loop using stored return address

4.5.6 NOP Instructions

NOP instructions are used as padding to ensure correct instruction alignment for subroutine calls.

4.6 Processor Execution Behavior

- The Program Counter continuously updated to fetch sequential instructions.
- The ALU performed repeated addition operations to generate Fibonacci terms.
- Register values were continuously updated to store previous, current, and next sequence values.
- Branch instructions controlled loop repetition until the iteration counter reached zero.
- JAL and JALR instructions handled subroutine calls and returns.
- Stack memory operations preserved register values during subroutine execution.
- Memory-mapped store instructions updated FPGA LED outputs with current Fibonacci values.

These operations verified correct interaction between the control path and datapath modules throughout program execution.

4.7 Waveform Analysis (Task C)

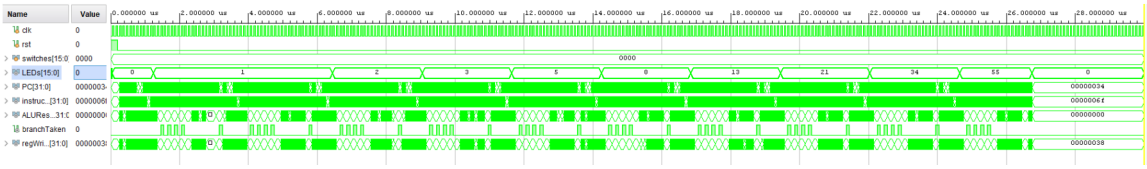

Figure 3: Task C Simulation Waveform

Explanation:

- PC loops through Fibonacci instructions
- Register values update correctly for each iteration
- ALU performs addition for Fibonacci computation
- LED output increases following Fibonacci sequence
- Subroutine calls and returns are visible through PC changes

4.8 Key Concepts Demonstrated

- Arithmetic progression
- Looping
- Stack frame management
- Subroutines
- JAL/JALR
- Memory-mapped display

4.9 Task C Summary

Task C successfully demonstrated execution of a complete assembly-level application on the custom RISC-V processor. The processor correctly generated Fibonacci values, handled arithmetic computation, controlled iterative branching, executed subroutines, and displayed outputs on FPGA hardware.

The task verified correct processor integration and demonstrated successful interaction between the ALU, register file, control unit, memory system, and FPGA I/O interfaces.

5 Conclusion

This project successfully demonstrated the design, implementation, and extension of a custom RISC-V single-cycle processor.

5.1 Major Achievements

- Integrated all processor modules into a complete functional architecture
- Successfully executed Task A countdown assembly program
- Implemented and verified SLT, BGE, and LUI instructions
- Executed Fibonacci sequence generation in Task C
- Demonstrated stack handling, subroutines, branching, and datapath extension
- Verified all tasks through:
  - Simulation
  - Waveforms
  - FPGA hardware outputs

5.2 Overall Learning Outcomes

This project provided practical understanding of:

- Processor datapath design
- Control path logic
- Instruction execution
- Assembly programming
- Hardware/software integration
- FPGA deployment

In conclusion, the project effectively combined theoretical processor concepts with practical hardware implementation, significantly strengthening understanding of computer architecture and RISC-V processor functionality.